Harpagon: Minimizing DNN Serving Cost via Efficient Dispatching, Scheduling and Splitting

Zhixin Zhao*, Yitao Hu*, Ziqi Gong*, Guotao Yang*, Wenxin Li*, Xiulong Liu*, Keqiu Li*, Hao Wang†,
*Tianjin Key Laboratory of Advanced Networking, Tianjin University, China
†Stevens Institute of Technology, USA

IEEE INFOCOM 2025 - IEEE Conference on Computer Communications | 979-8-3315-4305-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/INFOCOM55648.2025.11044536

***Abstract*—Advances in deep neural networks (DNNs) have significantly contributed to the development of real-time video processing applications. Efficient scheduling of DNN workloads in cloud-hosted inference systems is crucial to minimizing serving costs while meeting application latency constraints. However, existing systems suffer from excessive module latency during request dispatching, low execution throughput during module scheduling, and wasted latency budget during latency splitting for multi-DNN applications, which undermines their capability to minimize the serving cost.**

In this paper, we design a DNN inference system called Harpagon, which minimizes the serving cost under latency constraints with a three-level design. It first maximizes the batch collection rate with a batch-aware request dispatch policy to minimize the module latency. It then maximizes the module throughput with multi-tuple configurations and proper amount of dummy requests. It also carefully splits the end-to-end latency into per-module latency budget to minimize the total serving cost for multi-DNN applications. Evaluation shows that Harpagon outperforms the state of the art by 1.49 to 2.37 times in serving cost while satisfying the latency objectives. Additionally, compared to the optimal solution using brute force search, Harpagon derives the lower bound of serving cost for 91.5% workloads with millisecond level runtime.

I. Introduction

Deep neural network (DNN) has achieved state-of-the-art results in multiple fields [1], such as computer vision, speech recognition, and natural language processing. Therefore, the application developers are starting to build streaming applications using DNN models [2]–[6]. These DNN-based applications usually have latency constraints to guarantee good user experiences [7]–[11]. For example, the traffic monitoring application [12], which extracts pedestrian and vehicle information from videos collected by the surveillance cameras, often expects results to be returned within a few milliseconds to provide timely support when emergency occurs.

To satisfy the latency objective, the DNN-based applications are usually deployed in the inference system to leverage the plentiful GPU resources in the cloud [13]–[17]. The inference system uses the classic *client-server* architecture, where the client sends requests to the inference system to query the target DNN models with latency constraints. When providing services to the clients, the application developer is charged by the cloud provider according to the amount of computation resources used by the DNN-based application. Inference for DNN-based applications accounts for nearly 90% of computing costs in the cloud [18]. Therefore, when running the inference system, the primary goal for the application developer is to find a proper DNN module configuration (*e.g.,* the batch size for DNN inference), so as to minimize the serving cost while satisfying the latency objective.

However, existing serving systems [2]–[5] can *not* minimize the serving cost due to three limitations: (1) *Excessive module latency.* Existing systems dispatch requests among machines in individual request and form batched requests at the machine side, which has a relatively low batch collection rate and unnecessarily increases the module latency. (2) *Low module throughput.* Existing systems only support a fixed amount of configurations for each module and have limited support for resource heterogeneity, leading to relatively low throughput under the latency budget. (3) *Wasted latency budget.* Given the latency constraint for the entire application, existing systems either use quantized interval [2] or machine throughput [3], [4] to split the latency into per-module latency budget, leading to sub-optimal result. Therefore, these limitations result in low module throughput and eventually a higher serving cost for existing systems.

In this paper, we argue that **a proper DNN inference system should dispatch requests with higher batch collection rate, support flexible module configurations and assign each module a decent latency budget**, so as to maximize module throughput and thereby minimize the serving cost. Based on this, we design and implement a DNN inference system called Harpagon, which provides cost-minimum DNN inference services with efficient request dispatching, module scheduling and latency splitting capability as follows.

First, Harpagon designs a novel request dispatch method to minimize the latency of DNN modules by dispatching requests among machines in batched requests directly, which largely increases the batch collection rate and thereby reduces the module latency. By doing so, Harpagon is able to select module configurations with larger throughput under the latency constraint, leading to a lower serving cost.

Second, Harpagon leverages a novel module scheduling method to maximize module throughput. It uses a combination of batching and heterogeneous hardware to derive a multi-tuple configuration per module, which increases resource efficiency. Besides, for machines with small workload, Harpagon provides dummy generator and latency reassigner to increase

The work of Yitao Hu was supported by National Key Research and Development Program of China (2022YFB4501000) and National Natural Science Foundation of China under Grant Nos.62202328.

Corresponding author: Yitao Hu (email: yitao@tju.edu.cn).

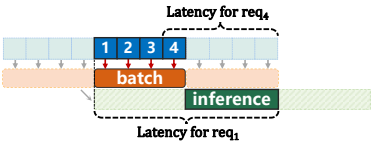

Fig. 1. Requests in the same batch have varying latency.

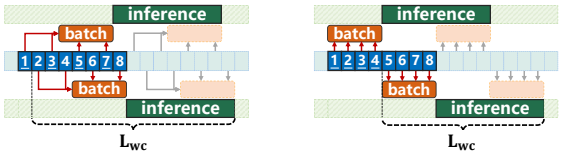

Fig. 2. (a) round-robin dispatch; (b) batch dispatch.

their cost efficiency, which further reduces the cost.

Third, Harpagon proposes a novel latency splitting method to minimize the total cost for multi-DNN applications. It leverages the latency-cost efficiency to measure the amount of cost that each module configuration can save per unit of latency budget, and designs a heuristic to split the latency to minimize the total cost for the entire application. Besides, Harpagon supports various algorithms to optimize the splitting results and thereby further minimize the cost.

Harpagon is completed implemented as a containerized system deployed on a cluster of 16 GPUs with a total of 23k lines of Python code. Evaluation shows that Harpagon is able to minimize the serving cost while satisfying the latency objective. Compared to Harpagon, existing systems [2]–[5] require an average of 49.3% to 137.2% extra serving cost. For certain workload, existing systems require up to 3.2 times the cost of Harpagon's. For each workload, we generate the optimal cost with brute force search and show that Harpagon derives the optimal one for more than 91% workloads with an average runtime of 5 milliseconds. In the ablation study, we quantify the importance of Harpagon's design choices.

II. Background and Motivation

In this section, we start with the scheduling policies of existing serving systems, followed by three design dimensions that distinguish Harpagon from prior work.

How do existing systems decide module scheduling? Two key factors affect module scheduling. First, batching is widely used to increase module throughput with efficient utilization of GPU's computation capability [2], [5], [19]. Requests in the same batch experience different latency. For example, as shown in Fig. 1, $\texttt{req}_1$ has a much larger latency than $\texttt{req}_4$. The serving system defines the maximum latency of all requests as the worst case latency $\texttt{L}_{\texttt{wc}}$ and guarantees that $\texttt{L}_{\texttt{wc}}$ is within the target latency objective. Second, among the available computation hardware in the cloud, the most cost-efficient hardware is module dependent [4], [20]. Therefore, the serving system needs to find the optimal module configuration (*e.g.,* batch size and computation hardware) to minimize the serving cost, while satisfying the latency objective.

To do so, existing systems [2]–[5] often rely on a two-round heuristic: (1) it first estimates the worst case latency

TABLE I
Module profiles, where b and d are batch size and execution duration (sec), t=b/d is module throughput (req/sec).

Module $\texttt{M}_1$			Module $\texttt{M}_2$			Module $\texttt{M}_3$		
b	d	t	b	d	t	b	d	t
2	0.160	12.5	2	0.125	16	2	0.100	20
4	0.200	20	4	0.160	25	8	0.250	32
8	0.320	25	8	0.250	32	32	0.800	40

$\texttt{L}_{\texttt{wc}}$ of all candidate configurations, and (2) then greedily select the optimal one that maximizes the throughput while its $\texttt{L}_{\texttt{wc}}$ is within the latency budget. For example, we consider a workload of module $\texttt{M}_1$ from Table I with a request rate of 100 req/sec and latency SLO of 0.4 sec. Existing systems dispatch individual requests among machines to form batches in a round-robin fashion as shown in Fig. 2(a), leading to a worst case latency two times of module duration (*i.e.,* $\texttt{L}_{\texttt{wc}} = 2\texttt{d}$) [2], [3]. Therefore, $\texttt{L}_{\texttt{wc}}$ for batch size of 2, 4 and 8 will be 0.32, 0.4 and 0.64 sec, and existing systems will select batch size of 4, since it maximizes module throughput while satisfying latency objective.

Request dispatching. We argue that the request dispatching policy of existing systems cannot minimize the worst case latency $\texttt{L}_{\texttt{wc}}$. As shown in Fig. 2(a), existing systems [2]–[5] dispatch requests one by one to each machine (*e.g.,* $\texttt{req}_{1,3,5,7}$) and form batched request at the machine side, so each machine receives requests at a rate equivalent to *its per-machine module throughput*, leading to $\texttt{L}_{\texttt{wc}} = 2\texttt{d}$.

However, if the serving system can dispatch requests among machines in batches directly, $\texttt{L}_{\texttt{wc}}$ can be largely reduced. As shown in Fig. 2(b), if we dispatch a consecutive amount of requests equal to its batch size to each machine (*e.g.,* $\texttt{req}_{1\text{-}4}$), each machine will receive requests at a rate equivalent to *the total request rate*, leading to $\texttt{L}_{\texttt{wc}} = \texttt{d} + \texttt{b}/\texttt{T}$, where b, d and T is the batch size, execution duration and total request rate.

By dispatching requests in batch, the serving system shifts the batch collection process from the machine to the frontend, which can potentially reduce the serving cost with a smaller $\texttt{L}_{\texttt{wc}}$. For example, in the above example of $\texttt{M}_1$, when dispatching in batches, $\texttt{L}_{\texttt{wc}}$ for batch size of 2, 4 and 8 will be 0.18, 0.24 and 0.4 sec. Now the serving system can choose batch size 8, which is infeasible for existing systems. Therefore, to handle 100 req/sec, serving systems with batch-aware dispatch only require $\texttt{n} = \texttt{T}/\texttt{t} = 100/25 = 4$ machines with batch size 8, while existing ones with round-robin dispatch require $\texttt{n} = 100/20 = 5$ machines with batch size 4.

***Key question* ❶:** Since the incoming request rate doesn't have to be an integer multiple of the module throughput of any given configuration, each module is usually associated with multiple configurations, as we will discuss next. Therefore, for each module, Harpagon needs to derive a proper request dispatch strategy across its configurations to minimize module's worst case latency.

Module scheduling. We argue that the module scheduling policy of existing systems cannot maximize module throughput under the latency budget. As discussed, existing systems [2]–

TABLE II

SCHEDULING RESULTS AND CORRESPONDING SERVING COST OF FOUR SCHEDULING METHODS S_1 - S_4 FOR M_3. EACH SCHEDULING RESULT INCLUDES K CONFIGURATIONS, WHERE THE ith CONFIGURATION T_i $(n_i \otimes b_i)$ MEANS THAT n_i MACHINE WITH BATCH SIZE OF b_i WILL DEAL WITH REQUEST RATE OF T_i req/sec, AND THE COST $= \sum_1^K n_i$.

Method	S_1	S_2	S_3	S_4
Dispatch	round-robin	batch-aware	batch-aware	batch-aware
#Config	2	2	any	any
Dummy	✗	✗	✗	✓
Config	192 $(6.0 \otimes 8)$ 6 $(0.3 \otimes 2)$	160 $(4.0 \otimes 32)$ 38 $(1.9 \otimes 2)$	160 $(4.0 \otimes 32)$ 32 $(1.0 \otimes 8)$ 6 $(0.3 \otimes 2)$	200 $(5.0 \otimes 32)$
Cost	6.3	5.9	5.3	5.0

[5] leverages a two-round greedy heuristic. The scheduler first chooses the optimal configuration c_{opt}, which derives the largest module throughput t_{opt} among all candidates while satisfying the latency objective. It allocates $n = \lfloor T/t_{opt} \rfloor$ machines running at c_{opt} for the majority workload $T_{maj} = n \cdot t_{opt}$. Then, the scheduler chooses another configuration c_{res} for the residual workload $T_{res} = T - n \cdot t_{opt}$ and allocates additional machine accordingly. Therefore, existing systems usually generate a two-tuple $\langle c_{opt}, c_{res} \rangle$ for each module.

For example, assuming a workload of module M_3 from Table I with request rate of 198 req/sec and latency SLO of 1.0 sec, Table II shows the scheduling results and corresponding serving costs for various scheduling methods: (1) Baseline S_1. Existing systems use round-robin dispatch and two-tuple configuration as S_1, which requires 6 machines at full capacity and 1 machine at partial capacity, leading to a serving cost of 6.3 machines. (2) S_1 to S_2. A better solution S_2, which dispatches batched requests, reduces the serving cost from 6.3 to 5.9 machines by reducing L_{wc} of candidate configurations. (3) S_2 to S_3. For certain workload, the serving cost can be further reduced with multi-tuple configurations. In the above example, S_3 keeps the majority workload unchanged and further splits the partial workload (*i.e.,* $38 \rightarrow 32 + 6$) to assign 2 machines with batch size 8 and 2 respectively, reducing the serving cost from 5.9 to 5.3 machines. Existing systems can *not* support multi-tuple configurations due to runtime complexity [2] or problem formulation [3]–[6]. (4) S_3 to S_4. We can surprisingly further reduce the serving cost by adding a proper amount of dummy requests. In the above example, if we add a dummy 2 req/sec (*i.e.,* 198 $\rightarrow$ 200 req/sec), S_4 reduces the serving cost from 5.3 to 5.0 machines. Specifically, though the dummy requests will consume a certain amount of computation resources, the serving cost can be reduced by increasing the batch size of those machines assigned to the residual workload.

Key question ❷: It is challenging to schedule modules in a complex configuration space. For example, naively adding dummy of 10 req/sec will only add extra workload without any cost reduction. Therefore, Harpagon needs to properly design module scheduling.

Latency splitting. Recent applications are starting to use multiple DNN modules to improve performance. For such applications, the serving system is responsible for splitting the end-to-end latency into per-module latency budgets. We argue that existing systems cannot utilize the latency budget efficiently.

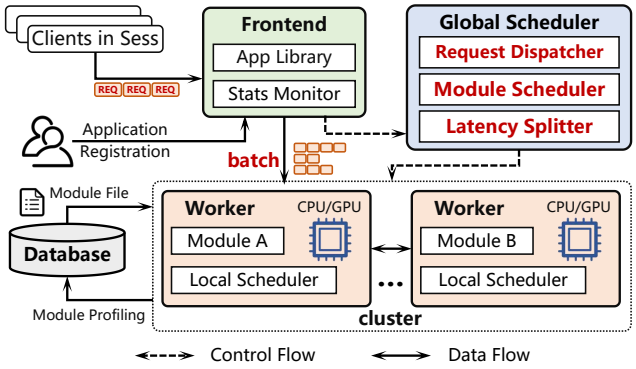

Fig. 3. Harpagon overview.

Deriving the optimal latency splitting strategy is known to be NP-Complete [2]–[4]. Therefore, existing systems mainly leverage two types of heuristics. The first one [2] divides the end-to-end latency into discretized intervals to reduce the search space. However, its optimality is decided by the interval length and its complexity is exponential to the number of modules and the reciprocal of interval length. The second one [3], [4] allocates latency budget across modules greedily according to module throughput. However, it prefers modules with larger throughput without consideration for its efficiency on latency budget, leading to sub-optimal solutions (§IV).

Key question ❸: Utilizing the end-to-end latency budget is critical to minimize the total cost for multi-DNN applications. Therefore, Harpagon needs to design a proper latency splitting strategy to derive per-module latency budget.

III. DESIGN

In this section, we start with an overview of Harpagon, followed by a detailed description of its components.

A. Overview

Terminology. Similar to existing works [2], [3], we define all requests for the same DNN-based application as a session. Each session has a unique *session id*, associated with (1) an application directed acyclic graph (DAG), where a node in the DAG represents a DNN module or processing module for GPU/CPU computation, and an edge represents computation dependency across nodes; (2) the request rate for each node in the DAG; and (3) a latency objective, which is the expected end-to-end latency for the entire application.

To capture the resource usage of modules, Harpagon provides a profiling library in the shared database for each module, which includes the execution duration of the given module under various configurations (*e.g.,* batch size and computation hardware). Similar to existing works [2]–[6], [21], the profiling is collected offline once when the application is registered, and will *not* affect the runtime latency of requests.

The cost is defined as the actual amount of computation resources used by the workload, according to frame-rate

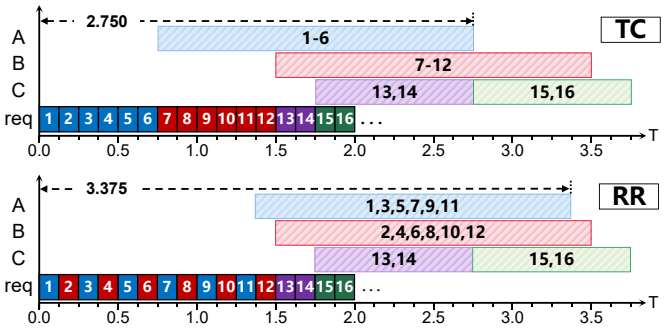

Fig. 4. Request dispatching results for throughput-cost dispatch policy (top) and round-robin dispatch policy (bottom).

proportionality [3]. Specifically, for a session with m modules, if the scheduler allocates N_i machines to handle module M_i, the session will have a cost of $\sum_{i=1}^{m}\sum_{j=1}^{N_i} p_{ij}\cdot f_{ij}/t_{ij}$, where p_{ij}, f_{ij} and t_{ij} is the unit price, the assigned request rate and the module throughput for the jth machine of M_i.

How Harpagon minimizes the serving cost. As shown in Fig. 3, for each session, the global scheduler decides the configuration of each module given its request rate and latency objective. Request dispatcher first derives the proper batch-aware dispatching strategy to minimize L_{wc}. Module scheduler and latency splitter then iteratively update candidate module configuration and per-module latency budget to minimize the total serving cost for the entire multi-DNN application.

Next, we first describe Harpagon's request dispatching method (§III-B), followed by its module scheduling method (§III-C), and finally the latency splitting method (§III-D).

B. Request Dispatching

To serve DNN-based applications, the first step is to estimate the worst case latency L_{wc} of all candidate configurations. As discussed, L_{wc} is closely related to request dispatch policy, and existing ones cannot minimize L_{wc}. Harpagon addresses the issue with its novel request dispatcher.

Throughput-Cost Request Dispatcher. Each candidate configuration is actually a set of configurations to handle a given module's majority and residual workload. For each set, Harpagon will rank all machines according to their configuration's throughput-cost ratio r, which is the module throughput divided by the hardware unit price. For example, for an incoming workload of 8 req/sec for module M_4, one of the candidate configuration set includes three machines A, B and C, where both A and B use configuration of batch size 6 with execution duration of 2.0 sec, and C uses configuration of batch size 2 with execution duration of 1.0 sec. All three machines have the same hardware unit price of 1.0. Harpagon calculates their throughput-cost ratio as $r_A = r_B = \frac{b}{d}/p = \frac{6}{2.0}/1.0 = 3.0$ and $r_C = \frac{2}{1.0}/1.0 = 2.0$, where b, d and p is the corresponding batch size, execution duration and hardware unit price. Therefore, Harpagon will rank these three machines as $r_A = r_B > r_C$.

Harpagon dispatches batched requests among machines in the order of throughput-cost ratio, where each machine will receive a successive amount of requests equal to its batch size in a row. For machines with the same throughput-cost ratio (*e.g.,* machines that use the same configuration such as A and B in the above example), Harpagon will dispatch requests among them in batched requests in turn. On the contrary, existing systems [2], [3], [5] dispatch requests among machines as individual request in round robin and let each machine collect the batch locally. We call Harpagon's dispatching strategy throughput-cost (TC) dispatch policy, as opposed to existing systems' round-robin (RR) dispatch policy. As discussed in §II, TC dispatch policy ensures that each machine collects requests to form the batch at the rate of the whole workload, while RR dispatch policy can only collect requests at the rate of machine's own module throughput.

By accelerating the batch collection rate, the TC dispatch policy can achieve a lower worst case latency than the RR dispatch policy. For example, given the above example of 8 req/sec for module M_4, Fig. 4 shows the request dispatching results using TC and RR dispatch policy, respectively. For the first 16 requests, the TC dispatch policy will dispatch $req_{1\text{-}6}$ to A, $req_{7\text{-}12}$ to B and $req_{13\text{-}16}$ to C, leading to a worst case latency of 2.75 sec with 0.75 sec for batch collection. Meanwhile, the RR dispatch policy will dispatch requests among A and B back and forth for $req_{1\text{-}12}$ without considering batch information, and then send $req_{13\text{-}16}$ to C, leading to a worst case latency of 3.375 sec with 1.375 sec for batch collection.

For a module, given a configuration set of n machines ordered by throughput-cost ratio: $r_1 \geq r_2 \geq \cdots \geq r_n$, we denote the configuration of machine i as c_i and the assigned request rate for machine i as f_i. The remaining workload for machine i is defined as $w_i = \sum_{j=1}^{n} s_{ij}\cdot f_j$, where s_{ij} is 1 if $r_j \leq r_i$ and is 0 if $r_j > r_i$. Namely, w_i represents the amount of request rate assigned to machines with throughput-cost ratio no larger than machine i's. For example, in the above example of M_4, the remaining workload for both A and B is $3+3+2=8$ req/sec, while the one for C is 2 req/sec.

Theorem 1. Using TC dispatch, the worst case latency for machine i is $L_{wc}(i) = d_i + b_i/w_i$, where d_i and b_i is the execution duration and batch size under configuration c_i. The worst case latency for the module is $\max_{i=1}^{n} L_{wc}(i)$.

Proof. TC dispatch policy sends batched requests among machines in the order of throughput-cost ratio, so machine i's remaining workload w_i is its equivalent batch collection rate, leading to a batch collection time $L_{col} = b_i/w_i$. Therefore, $L_{wc}(i)$ will be $d_i + b_i/w_i$, and the module's worst case latency will be the maximum of each machine's $L_{wc}(i)$. □

As shown in Theorem 1, Harpagon's TC dispatch can achieve a worst case latency of $d_i + b_i/w_i$ for each machine i, which is smaller than existing systems' $2d_i$ worst case latency using RR dispatch. By minimizing L_{wc} for all candidate configurations, Harpagon is able to choose configurations with larger throughput, which is infeasible for existing systems due to latency constraints, leading to a smaller serving cost (§IV).

Algorithm 1 Generating the configuration set for module M

```
1:  function GenerateConfig(T_M, L_M, P_M)
2:      rw ← T_M, configs ← []
3:      k ← 0, c ← P_M[k] = ⟨b_k, s_k, d_k, t_k, hw_k, p_k⟩
4:      while rw ≠ 0 do
5:          if GetWCL(c) ≤ L_M then
6:              n ← rw/t
7:              if n ≥ 1 then
8:                  configs ← configs ⊕ ⟨c, ⌊n⌋⟩
9:                  rw ← rw - ⌊n⌋ × t
10:             else
11:                 configs ← configs ⊕ ⟨c, n⟩
12:                 rw ← 0
13:         else
14:             while GetWCL(c) > L_M do
15:                 c ← P_M[++k]
16:                 if k ≥ len(P_M) then
17:                     return False, []
18:     return True, configs
```

C. Module Scheduling

Next, the DNN serving system needs to schedule modules given its latency budget and request rate. As discussed, existing systems use two-tuple configuration and cannot achieve high module throughput. Harpagon addresses the issue with its novel module scheduler and residual optimizer.

Module Scheduler. Given the worst case latency of all candidate configurations, Harpagon derives the multi-tuple configuration set for module M using Algorithm 1, where the input is the request rate $\mathtt{T_M}$, the latency budget $\mathtt{L_M}$ and the profiling $\mathtt{P_M}$ for all candidate configurations ordered by throughput-cost ratio (Line 1). Harpagon maintains the current unallocated workload rw, starting at $\mathtt{T_M}$ (Line 2). The configuration index k, starting at 0, records the current configuration c (Line 3). Function GetWCL() estimates $\mathtt{L_{wc}}$ as described in Theorem 1. Algorithm 1 generates the configuration set greedily. If the current configuration c can satisfy the latency budget (Line 5), Harpagon will use c to update configs. When the number of machines n is larger than 1, $\lfloor \mathtt{n} \rfloor$ machines will run at c at full capacity (Line 8) and corresponding amount of workload will be deducted from rw (Line 9). Otherwise, one machine will run at c at partial capacity of n (Line 11) and set rw to 0 (Line 12). If c can *not* satisfy the latency objective, Harpagon will find the next feasible configuration (Line 14-15).

Optimizing Residual Workload. Harpagon leverages its dummy generator and latency reassigner to increase module throughput for residual workload as follows.

Dummy generator adds a proper amount of dummy requests to the given module. Given a module with K distinct module configurations ordered by throughput-cost ratio: $\frac{\mathtt{t_1}}{\mathtt{p_1}} > \frac{\mathtt{t_2}}{\mathtt{p_2}} > \cdots > \frac{\mathtt{t_K}}{\mathtt{p_K}}$, where the ith module configuration $\mathtt{c_i}$ with module throughput $\mathtt{t_i}$ and unit price $\mathtt{p_i}$ is assigned $\mathtt{n_i}$ machines, Harpagon defines the *leftover workload* for $\mathtt{c_i}$ as $\mathtt{u_i} = \sum_{\mathtt{j=i+1}}^{\mathtt{K}} \mathtt{n_j t_j}$. Different from the remaining workload defined in §III-B, the leftover workload represents the total amount of requests assigned to machines with a throughput-cost ratio less than the given module configuration's.

Theorem 2. In the cost-minimum configuration, the leftover workload $\mathtt{u_i}$ for the ith configuration $\mathtt{c_i}$ ordered by throughput-cost ratio should be less than its throughput $\mathtt{t_i}$. Namely, $\forall \mathtt{i} \in \mathbb{K}, \mathtt{u_i} < \mathtt{t_i}$.

Proof. We use proof of contradiction, where $\exists k \in \mathbb{K}, u_k = \sum_{j=k+1}^{K} n_j t_j \geq t_k$. For u_k, we define the total number of machine as $n_\alpha = \sum_{j=k+1}^{K} n_j$, the average unit price as $p_\alpha = \sum_{j=k+1}^{K} n_j p_j / n_\alpha$, and the average throughput as $t_\alpha = u_k / n_\alpha$. The total cost for u_k is $C_0 = \sum_{j=k+1}^{K} n_j p_j$. Since $u_k \geq t_k$, we allocate a new machine at c_k to handle t_k workload for higher throughput-cost. Depending on the worst case latency for the remaining $u_k - t_k$ workload, there will be two cases.

In the first case, the original configuration for $u_k - t_k$ can still satisfy the latency constraint, then Harpagon will switch t_k amount of workload from its original configuration to c_k for higher throughput-cost efficiency and keep $u_k - t_k$ workload unchanged. We define $T_1 = \lfloor u_k / t_k \rfloor \cdot t_k$, then the new total cost for u_k is $C_1 = p_k \cdot \lfloor u_k / t_k \rfloor + p_\alpha \cdot (u_k - T_1)/t_\alpha$. We have $C_0 - C_1 = T_1(p_\alpha / t_\alpha - p_k / t_k)$. According to definition, $t_k / p_k > t_\alpha / p_\alpha$, so $C_0 - C_1 > 0$. Namely, the cost-minimum assumption is violated.

In the second case, the original configuration for $u_k - t_k$ can not satisfy the latency constraint, then Harpagon will choose a new configuration c_β to deal with the remaining $u_k - t_k$ workload to guarantee the latency SLO. Similarly, the new total cost for u_k is $C_2 = p_k \cdot \lfloor u_k / t_k \rfloor + p_\beta \cdot (u_k - T_1)/t_\beta$. We have $C_0 - C_2 = T_1(\frac{p_\alpha}{t_\alpha} - \frac{p_k}{t_k}) - (u_k - T_1)(\frac{p_\beta}{t_\beta} - \frac{p_\alpha}{t_\alpha})$. According to definition, $T_1 > u_k - T_1$, and for most modules, $\frac{p_\alpha}{t_\alpha} - \frac{p_k}{t_k} \approx \frac{p_\beta}{t_\beta} - \frac{p_\alpha}{t_\alpha}$, so $C_0 - C2 > 0$. Namely, the cost-minimum assumption is violated. □

Given Theorem 2, configuration $\mathtt{c_i}$'s leftover workload requires up to one extra machine at $\mathtt{c_i}$. Therefore, for a given module M with latency budget $\mathtt{L_M}$, Harpagon determines whether adding dummy request of $\mathtt{dum_i} = \mathtt{t_i} - \mathtt{u_i}$ can reduce the serving cost. For example, in the example of $\mathtt{M_3}$ with request rate of 198 req/sec in §II, according to the definition, $\mathtt{u_i}$ for module configuration with batch size 32 should be $\mathtt{u_i} = 32 + 6 = 38$ req/sec. A dummy request of $\mathtt{dum_i} = 40 - 38 = 2$ req/sec will reduce the serving cost to 5.0 machines, corresponding to scheduling result $\mathtt{S_4}$ in Table II.

Latency reassigner reassigns the remaining latency budget to module's residual workload. Harpagon derives the configurations using Algorithm 1. However, there can still be a gap between the worst case latency and the latency budget, which can be used to further reduce the serving cost when reassigned properly. Intuitively, this latency gap will *not* benefit the majority configuration for either module, otherwise Algorithm 1 should output another configuration result in the first place. Instead, reassigning the latency gap to module's residual workload can potentially increase its throughput. Harpagon re-runs Algorithm 1 with the updated latency budget for residual workload, and updates the configuration set if doing so can reduce the serving cost.

The intuition behind these two methods to increase the throughput for residual workload is associated with the latency constraint: $\mathtt{d} + \mathtt{b}/\mathtt{w} \leq \mathtt{L_M}$. Adding dummy requests is equiva-

Algorithm 2 Deriving per-module latency budget

```
1: function SplitLatency(T, L, P)
2:     DAG ← GetDefaultDAG()
3:     flag ← True
4:     while flag do
5:         flag ← False, LC_max ← −∞
6:         M_max ← NULL, c_max ← NULL
7:         for M in DAG do
8:             T_M ← T[M], P_M ← P[M]
9:             c_prev ← DAG[M]
10:            for c_new in P_M do
11:                LC ← (p_prev × T_M/t_prev − p_new × T_M/t_new) / (GetWCL(c_new) − GetWCL(c_prev))
12:                if LC > LC_max then
13:                    if GetLat(DAG, M, c_new) ≤ L then
14:                        flag ← True, LC_max ← LC
15:                        M_max ← M, c_max ← c_new
16:        if flag then
17:            updateDAG(DAG, M_max, c_max)
18:    return [L_M for M in DAG]
```

lent to increasing the remaining workload w, while reassigning the latency budget is equivalent to increasing the latency budget $\mathtt{L_M}$ for the residual workload, both of which enable Harpagon to choose configurations with larger throughput for the residual workload, leading to a lower serving cost (§IV).

D. Latency Splitting

Latency splitting derives the per-module latency budget. As discussed, existing ones often lead to sub-optimal results. Harpagon provides heuristics based on latency-cost efficiency to gradually allocate latency budget across modules and supports splitting optimizer to further reduce the total cost.

Latency Splitter. Harpagon defines the latency-cost efficiency LC as the amount of cost that a given module configuration can reduce per unit of latency budget when switching from the previous module configuration $\mathtt{c_{prev}}$ to the new one $\mathtt{c_{new}}$. Specifically, for a module with request rate of T, $\mathtt{LC} = \frac{\mathtt{C_M(prev)} - \mathtt{C_M(new)}}{\mathtt{L_{WC}(new)} - \mathtt{L_{WC}(prev)}}$, where $\mathtt{C_M}(*) = \mathtt{p}_* \times \mathtt{T}/\mathtt{t}_*$ and $\mathtt{L_{WC}}(*)$ is the serving cost and worst case latency for module M under the previous/new configuration. For example, for module $\mathtt{M_1}$ from Table I, we assume that the previous configuration is batch size of 2 to deal with a workload of $\mathtt{T} = 100$ req/sec and that $\mathtt{p}_*$ is 1.0. The other two configurations with batch size of 4 and 8 are the candidates to switch to. According to the definition, the latency-cost efficiency for configuration with batch size of 4 is $\frac{1.0\times100/12.5 - 1.0\times100/20}{(0.2+4/100)-(0.16+2/100)} = 50.0$, while the one for configuration with batch size of 8 is 18.2.

Based on LC, Harpagon uses Algorithm 2 to partition the latency into per-module latency budget. It uses DAG to store configurations for each module, starting at a default DAG (Line 2), where each module has batch size of 1 on hardware with the highest unit price, corresponding to the least cost-efficient configuration. Function `GetLat(DAG, M, c)` calculates the end-to-end latency when replacing module M in DAG with configuration c and keeping the rest of modules unchanged. Among all configurations of all modules, Algorithm 2 chooses the one whose LC (Line 11) is the maximum, while the latency constraint can be satisfied (Line 13), and then updates the configuration to the new one (Line 17). Algorithm 2 repeats the above process until no configuration can be updated while preserving the latency constraint.

In each iteration (Line 4-17), Harpagon prefers configurations with the largest latency-cost efficiency over the one with the largest throughput, which does *not* mean that Harpagon sacrifices throughput. Instead, by preferring latency-cost efficiency over throughput, Harpagon is able to gradually allocate the latency among modules in multiple iterations, which is more likely to achieve global optimal. On the contrary, throughput-based methods recklessly allocate the latency, which can easily fall into local optimal (§IV).

Optimizing the Splitting Result. Harpagon supports two methods to further optimize the splitting process.

Node merger merges modules sharing the same parent and children modules into a super-module and calculates their LC as a whole. The intuition is that modules sharing the same parent and children should have the same latency budget, so the latency splitting process should consider them as a whole. For example, we consider an application with three modules $\mathtt{M_x}$, $\mathtt{M_y}$ and $\mathtt{M_z}$, where the output of $\mathtt{M_x}$ is the input to $\mathtt{M_y}$ and $\mathtt{M_z}$. We assume the latency budget is only sufficient to update one module and LC to update these three modules is 20, 10 and 15. Using Algorithm 2, Harpagon will update $\mathtt{M_x}$ and keep $\mathtt{M_y}$ and $\mathtt{M_z}$ unchanged. However, if we merge $\mathtt{M_y}$ and $\mathtt{M_z}$ together, their total LC exceeds $\mathtt{M_x}$'s (*e.g.,* $10 + 15 > 20$), leading to a lower total cost.

Cost-direct reverses Algorithm 2's final R iterations and greedily selects the candidate operation that reduces the most cost. The intuition is that during the last iterations, the remaining latency budget is small, so using cost to select operation directly can achieve a lower cost. For example, we consider an application with two modules $\mathtt{M_x}$ and $\mathtt{M_y}$ in sequence, where the remaining latency budget is only sufficient to update one module. We assume that the candidate operation for $\mathtt{M_x}$ will consume 0.01 sec of latency budget with LC of 20, while the one for $\mathtt{M_y}$ will consume 0.1 sec of latency budget with LC of 10. Updating $\mathtt{M_y}$, though having a smaller LC (*i.e.,* $10 < 20$), can further reduce the total cost (*i.e.,* $0.1 \times 10 > 0.01 \times 20$).

IV. Evaluation

A. Methodology

Implementation. We have implemented all the features of Harpagon described in §III with roughly 8k lines of Python code for its core functionality, and an additional 15k lines for the app library. Each component of Harpagon runs in container for resource isolation and elastic scalability. Harpagon supports DNN modules trained by various frameworks including Tensorflow [22] and PyTorch [23]. Harpagon is deployed on a cluster with 8 P100 GPUs and 8 V100 GPUs to demonstrate its capability to minimize the cost for heterogeneous hardware. We provide extensible APIs to register new applications with less than 20 lines of code without modifying the code.

Workloads. For fair comparison, we choose five multi-DNN applications used in existing systems [2]–[6]: traffic [12] uses variants of SSD [24] to detect vehicle and pedestrian in traffic video, face uses PRNet [25] to detect facial keypoints, pose uses OpenPose [26] to recognize human poses,

TABLE III
COMPARISON OF SERVING SYSTEMS

System	Worst Case Latency	Num of Configurations	Batch	Hetero	Residual Optimization	Latency Split	Split Optimization
Harpagon	d + b/w	any	✓	✓	dummy + reassign	latency-cost efficiency	merge + direct
Nexus [2]	2d	2	✓			quantized interval	
Scrooge [3]	d + b/t	2	✓	✓		throughput-based	
InferLine [4]	2d	1	✓	✓		throughput-based	
Clipper [5]	2d	1	✓			evenly splitting	

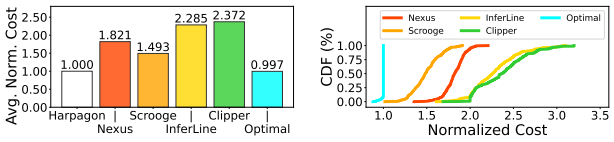

Fig. 5. (a) The average normalized cost for Harpagon, four baseline systems and the optimal solution under 1131 workloads; (b) CDF of normalized cost.

`caption` uses S2VT [27] to generate text description of video streams and `actdet` [28] detects human activities across camera footages. To demonstrate Harpagon's capability to minimize serving cost, we synthesize a total of 1131 workloads of the above five multi-DNN applications using public video streams [2], [27]–[29].

Comparison Baselines. We compare Harpagon against four open-sourced baseline systems: Nexus [2], Scrooge [3], InferLine [4] and Clipper [5], as well as the optimal solution derived from brute force search. Similar to Harpagon, all four baseline systems orchestrate DNN-based applications to minimize the cost while satisfying latency objectives. To serve multi-DNN applications, Nexus uses quantized intervals to split latency across modules, Scrooge and InferLine leverage module throughput to select configuration for each module. Clipper has limited support for multi-DNN applications, so we split latency across modules equally as in [2], [3].

Metrics. We define *the normalized cost* of a system as the ratio of the system's serving cost over Harpagon's. The ideal normalized cost should be as small as possible.

B. Comparison Results

Fig. 5(a) shows the average normalized cost for Harpagon, four baseline systems and the optimal solution under 1131 workloads, while Fig. 5(b) shows the cumulative distribution function (CDF) of normalized cost. Among five inference systems, Harpagon achieves the minimum serving cost for *all* workloads. Compared to Harpagon, the four baseline systems require an average extra cost of $49.3\% - 137.2\%$ and a maximum extra cost of $91.3\% - 220.0\%$. Table III shows the differences in design choices between Harpagon and the four baselines. Since the performance differences might come from a combination of multiple factors, we conjecture that the advantage of Harpagon comes from the following ones.

First, the request dispatching policy of the baseline systems can *not* minimize the worst case latency of candidate configurations. Harpagon's TC dispatch policy achieves a lower worst case latency than baseline systems', so Harpagon can leverage the saved latency to choose configurations with larger throughput or allocate more latency budget to other modules, both of which can reduce the serving cost.

Second, the module scheduling policy of the baseline systems can *not* maximize the module throughput under the given latency budget. Harpagon supports multiple configurations per module for higher resource efficiency, as opposed to baseline systems' fixed settings. Harpagon supports hardware heterogeneity to find proper computation hardware for each module. Besides, Harpagon leverages dummy generator and latency reassigner to increase module throughput of residual workload, which further reduces the serving cost.

Third, the latency splitting policy of the baseline systems can *not* utilize the latency efficiently. Early serving systems [5] have limited support for multi-DNN applications. Recent serving systems [2]–[4] use quantized interval or throughput-based splitting strategies, which have high runtime complexity or sub-optimal results. Harpagon leverages latency-cost efficiency to gradually allocate latency budget, and supports node merger and cost-direct to further minimize the total cost.

Besides, as shown in Fig. 5(b), compared to the optimal solution derived from brute force search, Harpagon's cost is larger than the optimal for *only* 8.5% workloads with up to 12.1% extra cost. However, the average runtime of brute force is 35.9 sec per workload, while Harpagon's runtime is *only* 0.005 sec. Namely, Harpagon derives the optimal for 91.5% workloads, while being more than 7000 times faster.

C. Ablation Study

In this section, we disable each of Harpagon's novel features to quantify its capability on cost minimization.

The importance of TC dispatch. Harpagon designs TC dispatch to minimize the worst case latency $\mathtt{L_{WC}}$ for all candidate configurations. To verify its capability to minimize the cost, we compare Harpagon against: (1) Harp-2d, which dispatches requests as individual ones [2], [4], [5], and (2) Harp-dt, which dispatches requests at rate of machine throughput [3].

Fig. 6 includes the normalized cost for Harp-2d and Harp-dt, which require 79.6% and 44.1% extra cost on average. As discussed in §III-B, TC dispatch minimizes $\mathtt{L_{WC}}$ with higher batch collection rate. To verify that, we choose configurations derived from Harp-2d under all 1131 workloads as configuration input to Harpagon, Harp-2d and Harp-dt for request dispatching. Fig. 7(a) shows the average normalized $\mathtt{L_{WC}}$. Given the *same* configuration, Harp-2d's RR dispatch policy and Harp-dt's throughput-based dispatch policy require an extra latency of 90.4% and 42.8% on average, indicating Harpagon's TC dispatch's capability to minimize $\mathtt{L_{WC}}$.

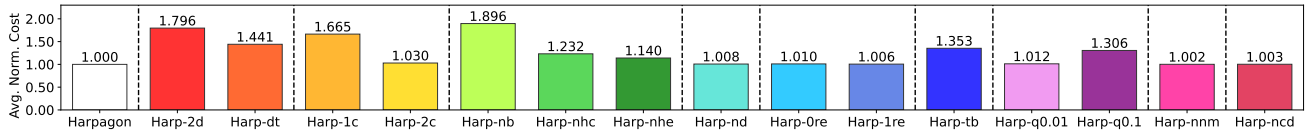

Fig. 6. The average normalized cost for Harpagon and baseline Harpagon without corresponding functionalities.

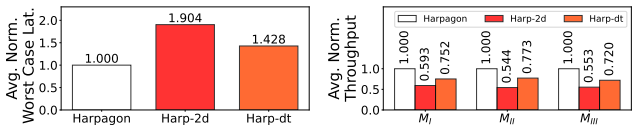

Fig. 7. (a) The average norm. worst case latency for Harp-2d and Harp-dt; (b) The average norm. throughput of three given modules.

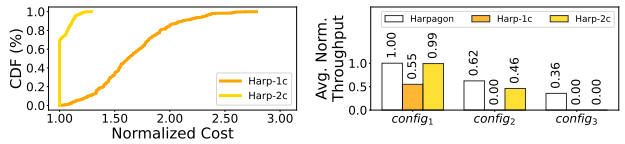

Fig. 8. (a) The CDF of normalized cost for Harp-1c and Harp-2c; (b) The average normalized throughput of configurations for Harp-1c and Harp-2c.

Moreover, to verify TC dispatch's capability on increasing throughput, Fig. 7(b) shows the average normalized throughput of three given modules, where the throughput for Harp-2d and Harp-dt is $40.7 - 45.6\%$ and $22.7 - 28.0\%$ lower than Harpagon's. The alternative systems have to choose batch sizes with smaller throughput due to their inefficient request dispatch policy. Meanwhile, with the minimum $\mathtt{L}_{\mathtt{WC}}$, Harpagon can choose batch sizes with larger throughput by leveraging the saved latency budget, leading to a smaller serving cost.

The importance of multiple configurations. Harpagon supports multiple configurations for each module to maximize the throughput. To quantify its benefit, we compare Harpagon against: (1) Harp-1c, which assigns each module one configuration [4], [5], and (2) Harp-2c, which assigns each module with two-tuple configurations [2], [3].

Fig. 6 includes the normalized cost for Harp-1c and Harp-2c, which require an average of 66.5% and 3.0% extra cost. Fig. 8(a) shows the CDF of the normalized cost. Harp-1c has a larger serving cost for almost *all* 1131 workloads and a maximum of 178.6% extra cost, while Harp-2c has the same cost as Harpagon for 67.6% workloads and a maximum of 29.0% extra cost. To understand why, Fig. 8(b) shows the average normalized throughput for a given module. The throughput for Harp-1c's sole configuration is 45.0% lower than Harpagon's. The throughput for Harp-2c's first configuration is almost the same as Harpagon's, but its second one is 26.1% lower than Harpagon's due to latency constraint. Among all workloads, Harpagon assigns 32.4% of them with more than two configurations to maximize the throughput, leading to a lower serving cost.

The importance of batching and heterogeneity. Harpagon selects proper batch size and computation hardware to minimize the serving cost. To quantify its importance, we compare Harpagon against: (1) Harp-nb, which disables batching, (2) Harp-nhc, which always selects the cheapest hardware, and (3) Harp-nhe, which selects the most expensive one.

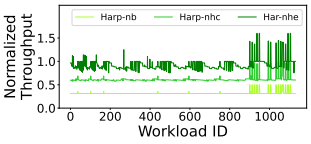

Fig. 9. The normalized throughput for Harp-nb, Harp-nhc and Harp-nhe.

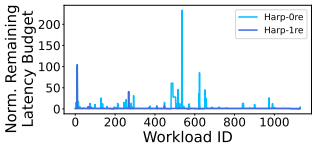

Fig. 10. The norm. remaining latency budget for Harp-0re and Harp-1re.

Fig. 6 includes the normalized cost for the three alternatives. Harp-nb has the highest serving cost with an average extra cost of 89.6%, suggesting the importance of batching on achieving the cost minimum goal. Harp-nhc and Harp-nhe require an average of 23.2% and 14.0% extra serving cost, which also indicates the necessity of selecting the proper hardware to reduce the cost. Both batching and heterogeneity reduce the serving cost by increasing the module throughput under the latency constraint. Fig. 9 shows the average throughput for a given module under 1131 workloads, where the throughput for the alternatives is 68.0%, 31.2% and 7.2% lower than Harpagon's. Interestingly, Harp-nhe's throughput for the given module is *larger* than Harpagon's among 4.9% workloads. This is because after disabling heterogeneity, Harp-nhe splits the latency differently, leaving the given module a larger latency budget. For all 4.9% workloads, the average throughput of Harp-nhe for the rest modules is much lower than Harpagon's, leading to a higher total cost.

The importance of optimizing residual. As discussed in §III-C, Harpagon adds dummy requests and reassigns the remaining latency budget to optimize the residual workload.

To quantify the benefit of adding dummy requests, we compare Harpagon against Harp-nd, which does not add dummy requests. Fig. 6 includes the normalized cost of Harp-nd, which requires 0.8% extra cost on average. For all 1131 workloads, Harpagon adds dummy request for 15.8% of them, among which dummy requests increase the throughput of residual workload by 80.8% with 10.9% extra computation resources, leading to a 4.8% cost reduction.

To quantify the benefit of reassigning the remaining latency budget, we compare Harpagon against: (1) Harp-0re, which does not reassign the latency budget, and (2) Harp-1re, which reassigns the latency budget to one module greedily. Fig. 6 includes the normalized cost for the two alternatives, which require an average of 1.0% and 0.6% and a maximum of 14.6% and 12.5% extra cost. Fig. 10 shows the remaining latency budget, where the one for Harp-0re and Harp-1re is

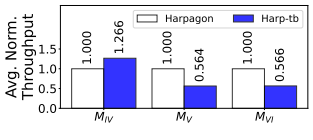

Fig. 11. The average normalized throughput for a three-module app.

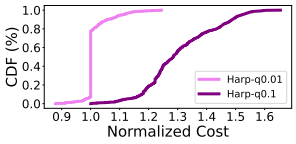

Fig. 12. The CDF of normalized cost for Harp-q0.01 and Harp-q0.1.

2.93 and 1.14 times of Harpagon's with a maximum of 233.3 and 104.3 times respectively. Among all workloads, Harpagon reassigns the remaining latency budget for at least once for 23.0% workloads to further reduce the cost.

The importance of latency-cost efficiency. Harpagon leverages latency-cost efficiency to split latency across modules. To quantify its benefit, we compare Harpagon against Harp-tb, which uses module throughput to split latency [3], [4].

Fig. 6 includes the normalized cost for Harp-tb, which requires an average of 35.3% extra cost. We measure the required iterations to derive the splitting results for Harpagon and Harp-tb. For all workloads, Harpagon uses 10.9 iterations on average, while Harp-tb only uses 3.2 iterations. As discussed in §III-D, Harpagon uses latency-cost efficiency to *gradually* allocate the latency budget across modules, which avoids achieving sub-optimal results as Harp-tb.

Besides, we find that the advantage of Harpagon over Harp-tb is larger for applications with more modules. Fig. 11 shows the average normalized throughput for a three-module application. Harp-tb assigns most of its latency budget to module $\mathtt{M_{IV}}$, so as to maximize the corresponding throughput. However, the remaining latency budget for the rest modules is limited. Namely, the alternative tends to allocate more latency budget to modules with higher throughput, which ignores the marginal cost reduction of latency budget. Meanwhile, Harpagon considers the amount of cost that per unit of latency budget can save, which enables Harpagon to minimize the cost, especially for applications with multiple modules.

The importance of not quantized. To derive per-module latency budget, an alternative is to quantize latency SLO into discrete intervals [2]. To show the benefit of Harpagon, we compare it against: (1) Harp-q0.01, which quantizes latency SLO into discrete interval in step of 0.01 sec, and (2) Harp-q0.1, which quantizes in step of 0.1 sec.

Fig. 6 includes the normalized cost for the two alternatives, while Fig. 12 shows the corresponding CDF. Harp-q0.1 requires an average of 30.6% and a maximum of 65.5% extra serving cost. This is because the 0.1-sec discrete interval is too coarse to split the latency SLO properly. Meanwhile, the serving cost of Harp-q0.01 is close to Harpagon's with an average of 1.2% and a maximum of 24.4% extra cost. Interestingly, for 7.3% workloads, the serving cost of Harp-q0.01 is smaller than Harpagon's. This is because the quantized method is another kind of brute force search. However, quantized-based solution is too slow. The average runtime of Harp-q0.01 is 2839 ms, while Harpagon's is only 5 ms. Namely, though Harp-q0.01 has lower cost for 7.3% workloads, it has higher cost for 23.3% workloads and a 567 times runtime complexity.

The importance of splitting optimization. Harpagon's latency-cost efficiency based method can *not* guarantee optimality, since latency splitting is NP-Complete. Harpagon supports two splitting optimizations. To quantify the benefit of node merging and cost direct, we compare Harpagon against: (1) Harp-nnm, which disables node merging, and (2) Harp-ncd, which disables cost direct. Fig. 6 includes the normalized cost for Harp-nnm and Harp-ncd, which requires an average of 0.2% and 0.3% extra serving cost. The alternatives benefit 3.98% and 5.07% workloads, among which they reduce 4.1% and 5.2% serving costs on average.

V. Related Work

DNN Inference Systems. Multiple DNN inference systems have been proposed recently, but Harpagon differs from them with its unique designs. Nexus [2] is designed to schedule DNN models under latency constraints. However, it uses round-robin dispatch policy, which can not achieve the cost-minimum goal. Scrooge [3] supports multi-dimensional scheduling, however, it only considers a maximum of two configurations and uses throughput-based heuristic to split the latency, leading to sub-optimal solution. InferLine [4] and Clipper [5] only consider one configuration per module. INFaaS [6] chooses appropriate DNN model among model variants by using a state machine to track variant performance, but it has limited support to split the latency for multi-DNN applications. Edge systems [21], [30]–[33] are designed for small clusters and cannot be applied for cloud directly.

DNN Training Systems. Gandiva [34] and AntMan [35] use spatial multiplexing and heterogeneous hardware to accelerate training process. Tiresias [36] and Themis [37] schedule training jobs in the cluster to minimize the job completion time. The design of Harpagon is inspired by various DNN training systems [38], [39], however, their scheduling capability can not satisfy inference job's millisecond latency objectives.

VI. Conclusion

In this paper, we presented a cost-minimum DNN inference system called Harpagon, which serves DNN workload efficiently to minimize the serving cost while satisfying the latency objectives. Harpagon dispatches batched requests across machines to minimize the worst case latency of candidate configurations, which enables it to choose configurations with larger throughput. Besides, Harpagon schedules modules with multiple configurations and leverages residual optimizer to maximize the throughput for both majority and residual workload. Moreover, for multi-DNN applications, Harpagon splits the end-to-end latency into per-module latency budget using latency-cost efficiency and supports splitting optimizer to maximize the latency efficiency. Evaluation shows that Harpagon outperforms the state of the art by 1.49 to 2.37 times in serving cost on average while satisfying the latency objective. Harpagon derives the lower bound cost for more than 91% workloads with millisecond-level runtime.

REFERENCES

[1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[2] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. Nexus: A gpu cluster engine for accelerating dnn-based video analysis. In *Proceedings of the 27th ACM SOSP*, pages 322–337, 2019.

[3] Yitao Hu, Rajrup Ghosh, and Ramesh Govindan. Scrooge: A cost-effective deep learning inference system. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 624–638, 2021.

[4] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. Inferline: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 477–491, 2020.

[5] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *NSDI*, volume 17, pages 613–627, 2017.

[6] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. Infaas: Automated model-less inference serving. In *USENIX Annual Technical Conference*, pages 397–411, 2021.

[7] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thinakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R Das. Cocktail: A multidimensional optimization for model serving in cloud. In *USENIX NSDI*, pages 1041–1057, 2022.

[8] Jing Wu, Lin Wang, Qirui Jin, and Fangming Liu. Graft: Efficient inference serving for hybrid deep learning with slo guarantees via dnn re-alignment. *IEEE Transactions on Parallel and Distributed Systems*, 2023.

[9] Jiabin Chen, Fei Xu, Yikun Gu, Li Chen, Fangming Liu, and Zhi Zhou. Harmonybatch: Batching multi-slo dnn inference with heterogeneous serverless functions. *arXiv preprint arXiv:2405.05633*, 2024.

[10] Haosong Peng, Yufeng Zhan, Peng Li, and Yuanqing Xia. Tangram: High-resolution video analytics on serverless platform with slo-aware batching. *arXiv preprint arXiv:2404.09267*, 2024.

[11] Rengan Dou, Xin Wang, and Richard TB Ma. Emma: Elastic multi-resource management for realtime stream processing. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024.

[12] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J Freedman. Live video analytics at scale with approximation and delay-tolerance. In *14th USENIX NSDI*, pages 377–392, 2017.

[13] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous gpu clusters. In *19th USENIX NSDI*, pages 945–960, 2022.

[14] Yongkang Zhang, Yinghao Yu, Wei Wang, Qiukai Chen, Jie Wu, Zuowei Zhang, Jiang Zhong, Tianchen Ding, Qizhen Weng, Lingyun Yang, et al. Workload consolidation in alibaba clusters: the good, the bad, and the ugly. In *Proceedings of the 13th Symposium on Cloud Computing*, pages 210–225, 2022.

[15] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. Transparent gpu sharing in container clouds for deep learning workloads. In *20th USENIX NSDI*, pages 69–85, 2023.

[16] Weimeng Wang, Lei Liu, and Zhongmin Yan. Proactive auto-scaling for delay-sensitive service providers over public clouds. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pages 01–09. IEEE, 2023.

[17] Fei Xu, Jianian Xu, Jiabin Chen, Li Chen, Ruitao Shang, Zhi Zhou, and Fangming Liu. igniter: Interference-aware gpu resource provisioning for predictable dnn inference in the cloud. *IEEE Transactions on Parallel and Distributed Systems*, 34(3):812–827, 2022.

[18] Reduce inference costs on Amazon EC2. https://aws.amazon.com/blogs/machine-learning/reduce-inference-costs-on-amazon-ec2-for-pytorch-models-with-amazon-elastic-inference.

[19] Zhaorui Wu, Yuhui Deng, Yi Zhou, Jie Li, and Shujie Pang. Faasbatch: Enhancing the efficiency of serverless computing by batching and expanding functions. In *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, pages 372–382. IEEE, 2023.

[20] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX OSDI*, pages 481–498, 2020.

[21] Yitao Hu, Weiwu Pang, Xiaochen Liu, Rajrup Ghosh, Bongjun Ko, Wei-Han Lee, and Ramesh Govindan. Rim: Offloading inference to the edge. In *Proceedings of the International Conference on Internet-of-Things Design and Implementation*, pages 80–92, 2021.

[22] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: a system for large-scale machine learning. In *OSDI*, pages 265–283, 2016.

[23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in NIPS*, 32, 2019.

[24] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. Ssd: Single shot multibox detector. In *ECCV*, pages 21–37. Springer, 2016.

[25] Yao Feng, Fan Wu, Xiaohu Shao, Yanfeng Wang, and Xi Zhou. Joint 3d face reconstruction and dense alignment with position map regression network. In *Proceedings of the European conference on computer vision (ECCV)*, pages 534–551, 2018.

[26] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7291–7299, 2017.

[27] Subhashini Venugopalan, Marcus Rohrbach, Jeffrey Donahue, Raymond Mooney, Trevor Darrell, and Kate Saenko. Sequence to sequence-video to text. In *Proceedings of the IEEE international conference on computer vision*, pages 4534–4542, 2015.

[28] Xiaochen Liu, Pradipta Ghosh, Oytun Ulutan, BS Manjunath, Kevin Chan, and Ramesh Govindan. Caesar: cross-camera complex activity recognition. In *Proceedings of the 17th Conference on Embedded Networked Sensor Systems*, pages 232–244, 2019.

[29] https://www.earthcam.com/, 2023.

[30] Yifan Zeng, Ruiting Zhou, Lei Jiao, Ziyi Han, Jieling Yu, and Yue Ma. Efficient online dnn inference with continuous learning in edge computing. In *2024 IEEE/ACM 32st International Symposium on Quality of Service (IWQoS)*. IEEE, 2024.

[31] Yechao She, Minming Li, Yang Jin, Meng Xu, Jianping Wang, and Bin Liu. On-demand edge inference scheduling with accuracy and deadline guarantee. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pages 1–10. IEEE, 2023.

[32] Liang Wang, Xiaoyang Qu, Jianzong Wang, Guokuan Li, Jiguang Wan, Nan Zhang, Song Guo, and Jing Xiao. Gecko: Resource-efficient and accurate queries in real-time video streams at the edge. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, 2024.

[33] Tao Wang, Tuo Shi, Xiulong Liu, Jianping Wang, Bin Liu, Yingshu Li, and Yechao She. Minimizing latency for multi-dnn inference on resource-limited cpu-only edge devices. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024.

[34] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.

[35] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. Antman: Dynamic scaling on gpu clusters for deep learning. In *OSDI*, pages 533–548, 2020.

[36] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A gpu cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, 2019.

[37] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient gpu cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, 2020.

[38] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael M Swift. Atp: In-network aggregation for multi-tenant learning. In *NSDI*, volume 21, pages 741–761, 2021.

[39] Yihao Zhao, Yuanqiang Liu, Yanghua Peng, Yibo Zhu, Xuanzhe Liu, and Xin Jin. Multi-resource interleaving for deep learning training. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 428–440, 2022.